

Exercises: Introductory Course on Implicitly Switched ODEs and Filippov Sliding

Andreas Sommer, Lev Gromov, Marvin Hertweck, Pilar Keller

July 09, 2026

Task 1: Introductory Example

Consider the following switched ODE

$$\begin{aligned}\frac{d}{dt}y_1(t) &= \begin{cases} q_1(t), & \text{if } y_1(t) \leq 750 \\ q_1(t) - q_2(t), & \text{if } y_1(t) > 750 \end{cases} \\ \frac{d}{dt}y_2(t) &= \begin{cases} 0, & \text{if } y_1(t) \leq 750 \\ -300, & \text{if } 750 < y_1(t) < 800 \\ 0, & \text{if } y_1(t) \geq 800 \end{cases}\end{aligned}\tag{1}$$

where

$$\begin{aligned}q_1(t) &= \frac{50}{27}t^2 - \frac{100}{3}t + \frac{400}{3} \\ q_2(t) &= (1000 - y_2(t))(p_1(t - 19) + p_2)\end{aligned}\tag{2}$$

a) Subtask:

Use `ode45` to solve this system on the time interval $I = [0, 30]$ with initial conditions $y_1(0) = 0$ and $y_2(0) = 1000$. Use $p = (\frac{3}{4}, \frac{1}{4})$ as the parameter vector.

Plot the solution. Does the solution look correct? Explain why `ode45` produces this result.

Steps:

1. Implement the RHS in the file `rhsTask1.m` as described by eq. (1) and eq. (2). Compute the values of $q_1(t)$ and $q_2(t)$ directly in the RHS as local variables `q1` and `q2` (do not implement them as separate functions). Use an `if condition1 ... elseif condition2 ... else ...` structure to formulate the three possible branches of the RHS.
2. Set the integration time span, initial conditions and parameters in the file `runTask1.m`.
3. Use `ode45` to compute a solution. Remember to pass the RHS as an anonymous function `@(t, y)rhsTaks1(t, y, p)`, which takes a timepoint and state vector and returns the derivative vector. Don't forget to provide the integration time span and initial conditions.
4. Create a vector of timepoints where you will plot the solution using `t = linspace(tspan(1), tspan(end), 1000)`.
5. Evaluate the solution at the timepoints using `y0de45 = deval(solution0de45, t)`.
6. Plot the solution using `plot(t, y0de45)`, which will plot both solution components into one plot.

b) Subtask:

Solve the system from subtask a) again, but this time with a maximum integrator step size of 0.1.

Plot the solution. Does the solution look correct? Explain why `ode45` produces this result.

Steps:

1. Add the maximum step size to the integrator options using `options = odeset(..., 'MaxStep', 0.1)`.
2. Run your code from subtask a) with the new options.

c) Subtask:

Use IFDIFF to solve the system from subtask a).

Plot the solution. Does the solution match the solution computed in subtask b)?

Steps:

1. Use IFDIFF's `prepareDatahandleForIntegration` function to prepare the RHS for integration. Don't forget to include the integrator and its options as optional arguments.
2. Use IFDIFF's `solveODE` function to solve the initial value problem.

3. Create a vector of timepoints where you will plot the solution using
`t = linspace(tspan(1), tspan(end), 1000).`
 4. Evaluate the solution at the timepoints using `yIfdiff = deval(solutionIfdiff, t).`
 5. Plot the solution using `plot(t, yIfdiff)`, which will plot both solution components into one plot.
- d) We will now examine how the solution behaves under changes to the initial values, which is described by the initial value sensitivity matrix

$$G_y(t; y_0) = \frac{d}{dy_0} y(t; y_0) \quad (3)$$

where $y(t; y_0)$ is the solution of the system described by eq. (1) and eq. (2) for initial values y_0 at timepoint t . The entry $(G_y(t; y_0))_{i,j}$ in the i -th row and j -th column of the matrix describes how the i -th solution component changes when the initial value of the j -th solution component is disturbed.

Subtask: Compute the initial value sensitivities for the solution computed by `ode45` in subtask b) by solving the Variational Differential Equations (VDE). Plot the result. Do the sensitivities look correct?

Steps:

1. Create a vector of timepoints for which you will compute and plot the sensitivities using
`t = linspace(tspan(1), tspan(end), 1000).`
 2. Use the provided function `sensitivityOde45 = computeSensitivityOde45(solutionOde45, t)` to compute the sensitivities for the solution generated by `ode45`.
 3. Use the provided function `plotSensitivity(sensitivityOde45)` to plot the sensitivities.
- e) **Subtask:**
 Use IFDIFF to compute the initial value sensitivities.
 Plot the result. Do the sensitivities look correct?

Steps:

1. Create a vector of timepoints for which you will compute and plot the sensitivities using
`t = linspace(tspan(1), tspan(end), 1000).`
2. Use IFDIFF's `generateSensitivityFunction` function to create a function `sensitivityFunction` which takes a vector of timepoints and returns the sensitivities at the given timepoints.
3. Call the sensitivity function with `sensitivityIfdiff = sensitivityFunction(t)` to compute the sensitivities at the timepoints.
4. Use the provided function `plotSensitivity(sensitivityIfdiff)` to plot the sensitivities.

For reference, your solution and sensitivity plots with IFDIFF should look like the following:

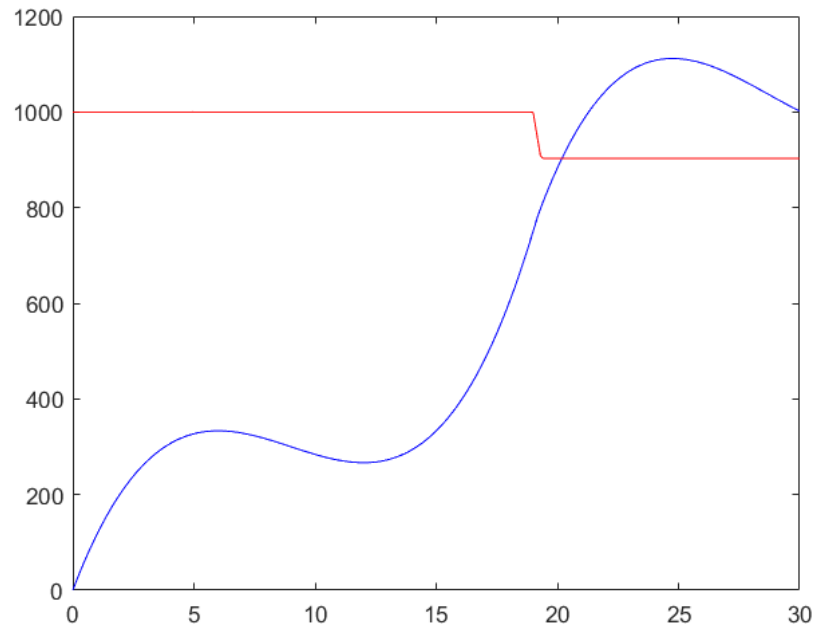


Figure 1: Solution plot for c)

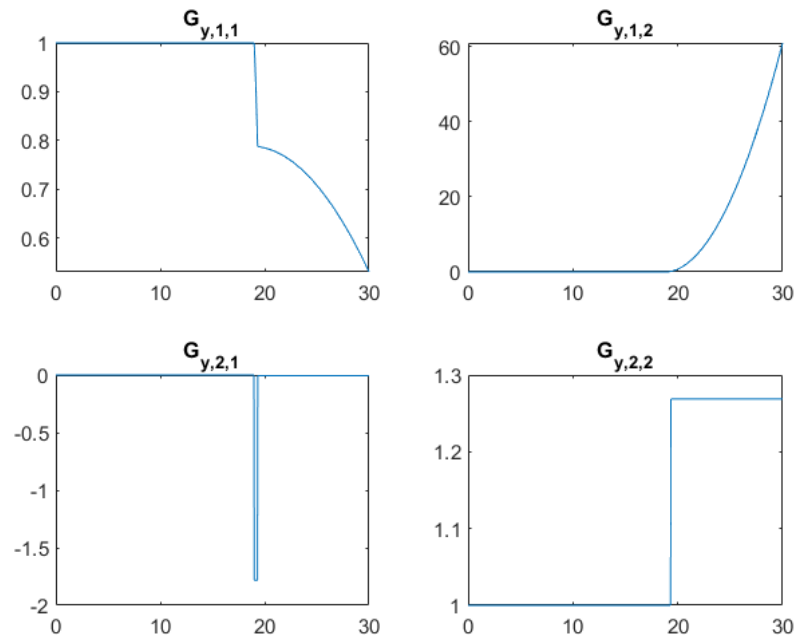


Figure 2: Sensitivities for d)

Appendix B: IFDIFF Cheat Sheet

RHS Function Requirements

- Signature: `function dx = myRHS(t, x, p)`. Keep in mind that the function file also needs to be named `myRHS.m`
- Helper functions work when defined either as:
 - in separate function files
 - as local variables
 - nested functions

but **DO NOT** work as local functions of `myRHS.m` !

- **Warning:** Do not use `elseif` statements, instead use nested `if ... else ... end` statements to code the switching behavior.

IFDIFF Solution Setup: A Canonical Example

We consider the following ODE:

$$\dot{x} = f(t, x, p) = \begin{pmatrix} f_1(t, x, p) \\ f_2(t, x, p) \end{pmatrix}$$
$$f_1(t, x, p) = 0.01 \cdot t^2 + x_2^3 \quad f_2(t, x, p) = \begin{cases} 0 & \text{if } x_1 < p \\ 5 & \text{if } p \leq x_1 < p + 0.5 \\ 0 & \text{if } x_1 \geq p + 0.5 \end{cases}$$

with the initial value $x(0) = (0, 1)^T$ and parameter $p = 5.437$

1. Setup the the RHS and name the file `canonicalExampleRHS.m`

```
function dx = canonicalExampleRHS(t,x,p)

dx = zeros(2,1);
dx(1) = 0.01 * t.^2 + x(2).^3;
if x(1) < p(1)
dx(2) = 0;
else
if x(1) < p(1) + 0.5
dx(2) = 5;
else
dx(2) = 0;
end
end

end
```

2. **Main Script:** Setup the initial values, the time interval as well as parameter(s):

```
t0 = 0;
tf = 20;
timeinterval = [t0,tf];
initstates   = [1 0];
p            = 5.437;
```

3. Define options in the main script and call `prepareDatahandleForIntegration` to prepare the right-hand side with the options for IFDIFF:

```
integrator = @ode45;
odeoptions = odeset( 'AbsTol', 1e-10, 'RelTol', 1e-5);
filename = 'canonicalExampleRHS';
datahandle = prepareDatahandleForIntegration(filename, 'integrator',
    integrator, 'options', odeoptions);
```

4. Create a solution structure using `solveODE` and evaluate

```
T = t0:0.1:tf;
sol_ifdiff = solveODE(datahandle, timeinterval, initstates, p);
Y_ifdiff = deval(sol_ifdiff, T);
```

5. Plot the solution

```
plot(T, Y_ifdiff);
xlabel('Time (s)');
ylabel('State Variables');
title('IFDIFF Solution');
grid on;
```

6. Compute and plot the sensitivities. There are two methods to do so. For the first one, you need to use `generateFDstep` so get finite differences and then use `generateSensitivityFunction` (which uses the VDE method in this example) to compute the sensitivities. Afterward, you can access and plot the sensitivities by appropriately reshaping the matrices in the `sensitivities` struct array, which are stored in the fields `Gy` for sensitivities w.r.t. initial values and `Gp` for sensitivities w.r.t. parameters respectively.

```
%% Step 5: Compute sensitivities manually

dim_y = size(Y_ifdiff, 1);
dim_p = length(p); % dim_p = 1

FDstep = generateFDstep(dim_y, dim_p, 'hy_rel_flag', true, 'hp_rel_flag', true, 'hy_min', 1e-6, 'hp_min', 1e-6);
sensFun = generateSensitivityFunction(datahandle, sol_ifdiff, FDstep, 'method', 'VDE', 'integrator_options', options);

sensitivities = sensFun(T); % Compute sensitivities at time points in T
```

Alternatively, you may use the provided `plotSensitivities_ifdiff` function which already computes the sensitivities internally and just outputs a plot:

```
plotSensitivities_ifdiff(datahandle, sol_ifdiff, p);
```

Your plots should look like the following:

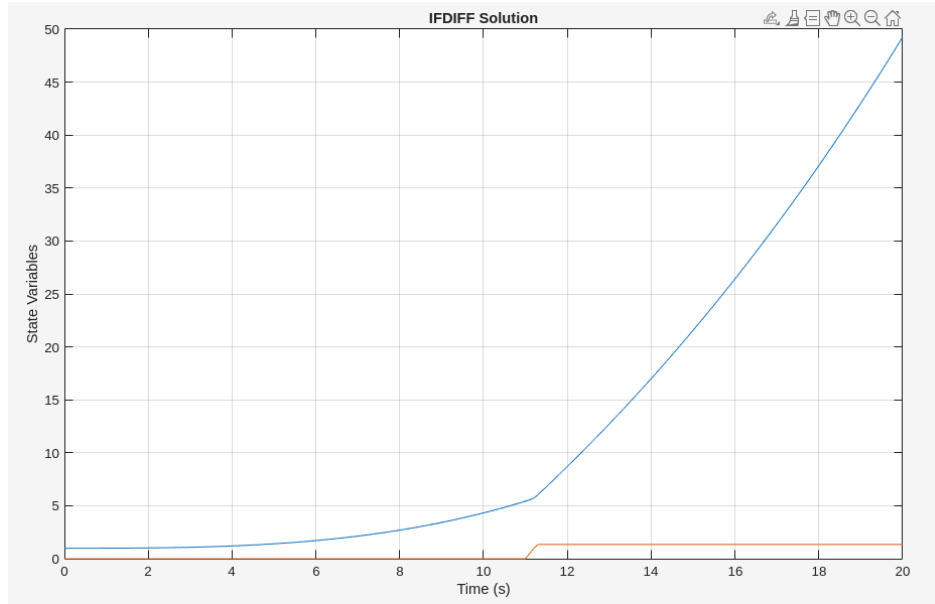


Figure 3: Canonical Example solution with IFDIFF

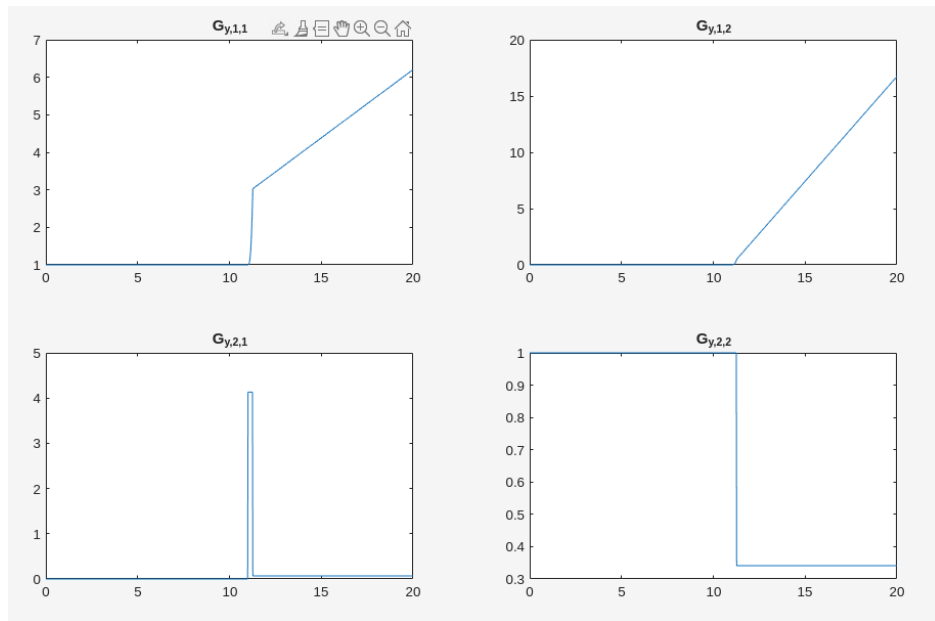


Figure 4: Sensitivities for the Canonical Example with IFDIFF

Specifying state jumps in a RHS

A state jump may be specified at any point in the RHS via a special if-block:

```
function dx = myRHS(t, x, p)
...
if ifdiff_jumpif(jumpCondition, jumpDirection)
... <statements computing the jumpIncrement> ...
endifdiff_update(jumpIncrement)
end
...
end
```

When the first argument of the `ifdiff_jumpif` function (`jumpCondition`) crosses zero in the direction specified by the second argument (`jumpDirection`), i.e. negative to positive (1), positive to negative (-1) or both (0), IFDIFF will increment the state vector in the solution by the first argument of the `ifdiff_update` function (`jumpIncrement`).

Warning: The state vector and `jumpIncrement` must have matching dimensions. In particular, the `jumpIncrement` must be a column-vector with the same number of elements as the state vector.

Warning: IFDIFF will never execute the body of the if-block directly in the RHS, so make sure that the statements in the if-block are only used to compute the `jumpIncrement`. In particular, any variables exclusively defined in the if-block must not be used outside of the if-block.

Plotting state jump sensitivities

Solve the system, compute the sensitivities and then call the `plotSensitivitiesSwitched` function.

Here is an example:

```
% RHS params
tspan = [0, 10];
h0 = 0;
v0 = 10;
x0 = [h0; v0];
p = 9.8;
% Solver params
solver = @ode45;
options = odeset('AbsTol', 1e-8, 'RelTol', 1e-6);
% Call IFDIFF
dh = prepareDatahandleForIntegration(@bounceRHS, 'integrator', solver, 'options', options);
sol = solveODE(dh, tspan, x0, p);

dim_y = size(sol.y, 1);
dim_p = length(p);
FDstep = generateFDstep(dim_y, dim_p);
sensFun = generateSensitivityFunction(dh, sol, FDstep, 'method', 'VDE');

plotSensitivitiesSwitched(sol, sensFun, tspan);
```